

Step 2 — A Real Terminal Chat With Memory and Live Streaming — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — Step 2: the model remembers nothing between calls (Doc03). So the app keeps a growing `history` list and resends it every turn — that list *is* the memory. Streaming makes the reply appear as it's written. Trimming keeps a long chat under the model's limit. **If not:** every reply would forget the previous one, the user would stare at a blank line while the whole reply is generated, and a long chat would eventually crash with a 400 error.

After this step the project is a working chat app. Files from Step 1 that aren't shown here (`config.py` , `exceptions.py` , `utils/logger.py`) don't change.

Basic Version

Approach 1 — the direct way

Story — `models/llm.py` (Basic): add Doc04's `stream_message()` next to Step 1's functions. **If not:** the loop would have to wait for each full reply.

```
# src/supportdesk/models/llm.py
import openai
from openai import OpenAI

from supportdesk.config import Config

MODEL = "gpt-4o-mini"

def create_client(config: Config) -> OpenAI:
    return OpenAI(api_key=config.openai_api_key)

def check_api_key_at_startup(client: OpenAI) -> bool:
    try:
        client.chat.completions.create(
            model=MODEL,
            messages=[{"role": "user", "content": "ping"}],
```

```

        max_tokens=1,
    )
except openai.AuthenticationError:
    return False
return True

def send_message(client: OpenAI, history: list[dict], user_input: str) -> str:
    history.append({"role": "user", "content": user_input})
    response = client.chat.completions.create(model=MODEL, messages=history)
    reply = response.choices[0].message.content
    history.append({"role": "assistant", "content": reply})
    return reply

def stream_message(
    client: OpenAI, history: list[dict], user_input: str
) -> str:
    history.append({"role": "user", "content": user_input})
    stream = client.chat.completions.create(
        model=MODEL, messages=history, stream=True
    )
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="", flush=True)
            full_reply = full_reply + piece
    print()
    history.append({"role": "assistant", "content": full_reply})
    return full_reply

```

Story — `main.py` (Basic): a loop around `stream_message()`, with the system prompt typed inline. **If not:** there'd be no conversation — just one message.

```

# src/supportdesk/main.py
import sys

from supportdesk.config import load_config
from supportdesk.exceptions import MissingConfigError
from supportdesk.models.llm import (
    check_api_key_at_startup,
    create_client,
    stream_message,
)

```

```

def main() -> None:
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    client = create_client(config)
    if not check_api_key_at_startup(client):
        print("Login failed – check OPENAI_API_KEY in your .env file.")
        sys.exit(1)

    history = [{"role": "system", "content": "You are a helpful assistant."}]
    while True:
        message = input("You: ")
        if message == "/quit":
            break
        print("Assistant: ", end="", flush=True)
        stream_message(client, history, message)

if __name__ == "__main__":
    main()

```

This works and passes the memory test. But `history` grows forever, so a long chat will eventually be rejected by the model. The prompt is buried in code, and an empty line still sends a request.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — prompt file, token budget, and a unit test

Changed from Step 1: one new function at the bottom, `stream_message()`. Everything above it is Step 1's code, unchanged.

Story — `models/llm.py`: Step 1's file plus Doc04's `stream_message()`. The one change from Doc04 is `flush=True`. **If not:** the terminal holds printed text back until the line ends, so the "stream" would appear all at once at the end.

```
# src/supportdesk/models/llm.py
import openai
from openai import OpenAI

from supportdesk.config import Config

# why: one place to change the model for every agent in the project
MODEL = "gpt-4o-mini"

def create_client(config: Config) -> OpenAI:
    # why: the key comes from the one validated place (load_config()),
    # not from the SDK quietly reading the environment by itself.
    # how: the client already retries rate limits, timeouts and 5xx
    # errors 2 times with backoff (Doc04) – so no retry loop of our own.
    return OpenAI(api_key=config.openai_api_key)

def check_api_key_at_startup(client: OpenAI) -> bool:
    # why: a bad key should fail once, clearly, before the user types
    # anything – not as a crash in the middle of their first message.
    # how: a 1-token reply costs almost nothing, and it tests the exact
    # endpoint every agent in this app depends on.
    try:
        client.chat.completions.create(
            model=MODEL,
            messages=[{"role": "user", "content": "ping"}],
            max_tokens=1,
        )
    except openai.AuthenticationError:
```

```

        return False
    return True

def send_message(client: OpenAI, history: list[dict], user_input: str) -> str:
    # why: a plain, non-streaming call – the tests use it, because they
    # need the finished reply text, not text printed piece by piece.
    history.append({"role": "user", "content": user_input})
    response = client.chat.completions.create(model=MODEL, messages=history)
    reply = response.choices[0].message.content
    history.append({"role": "assistant", "content": reply})
    return reply

# new in Step 2: everything below this line
def stream_message(
    client: OpenAI, history: list[dict], user_input: str
) -> str:
    # why: the chat reply appears word by word, like a real chat app.
    history.append({"role": "user", "content": user_input})
    stream = client.chat.completions.create(
        model=MODEL, messages=history, stream=True
    )
    full_reply = ""
    for chunk in stream:
        # how: each chunk carries one small piece of the reply; None means
        # this particular chunk carried no new text.
        piece = chunk.choices[0].delta.content
        if piece:
            # why: flush=True shows each piece right away – without it
            # the terminal holds the text back until the line ends.
            print(piece, end="", flush=True)
            full_reply = full_reply + piece
    print()
    # why: the reply goes into history, so the next turn remembers it
    history.append({"role": "assistant", "content": full_reply})
    return full_reply

```

Story — `prompts/concierge.txt` : the assistant's personality, as plain text. (It's called "concierge" already, because in Step 4 this chat becomes the Concierge agent.) **If not:** changing the tone would mean editing Python code.

`src/supportdesk/prompts/concierge.txt`

You are the SupportDesk concierge. Be friendly and brief.
Answer general questions and small talk about the service.

Story — `utils/prompts.py` : one function that reads a prompt file by name, so no agent opens files itself. **If not:** every agent would repeat the same `open(...)` code, each with its own copy of the folder path.

```
# src/supportdesk/utils/prompts.py

# how: a path from the repo root – every command in this project is run
# from the repo root, the same rule the tests and scripts follow
PROMPTS_DIR = "src/supportdesk/prompts"

def load_prompt(name: str) -> str:
    # why: prompts live in .txt files, not inside Python strings – you
    # can reword an agent's behavior without touching its code.
    # when: called whenever an agent needs its system prompt.
    # how: e.g. "concierge" -> "src/supportdesk/prompts/concierge.txt"
    path = PROMPTS_DIR + "/" + name + ".txt"
    with open(path) as prompt_file:
        # how: strip() drops the blank line at the end of the file
        return prompt_file.read().strip()
```

Story — `utils/history.py` : estimates the size of the history (Doc03's "about 4 characters per token") and drops the oldest turns until it fits a budget. The system prompt always stays. **If not:** a long chat would hit the context limit and fail with a 400 error mid-conversation — or, if you dropped from the front blindly, the assistant would lose its personality.

```
# src/supportdesk/utils/history.py
# why: the model has a hard context limit. Checking the size BEFORE
# sending means a long chat gets trimmed quietly, instead of failing
# with a 400 error in the middle of a conversation.

# how: Doc03's rule of thumb – about 4 characters per token in English
CHARS_PER_TOKEN = 4

def estimate_tokens(history: list[dict]) -> int:
    # why: a cheap guess, no extra library – good enough when the budget
    # sits far below the real limit
    total_chars = 0
    for message in history:
        total_chars = total_chars + len(message["content"])
    return total_chars // CHARS_PER_TOKEN

def trim_history(history: list[dict], max_tokens: int) -> list[dict]:
```

```
# why: history[0] is the system prompt (the agent's personality), so
# it is never dropped – the OLDEST real turn at history[1] goes first.
# how: len(history) > 2 always keeps the system prompt plus the
# newest message, even if that one message is over the budget alone.
while estimate_tokens(history) > max_tokens and len(history) > 2:
    del history[1]
return history
```

Changed from Step 1: the startup part of `main()` is the same. What's new: 2 imports, `MAX_HISTORY_TOKENS`, and the loop that replaces the single `send_message()` call.

Story — `main.py`: Step 1's startup, then a loop. It skips empty lines, trims before every call, and loads the system prompt from its file. **If not:** an empty Enter would cost an API call, and nothing would stop the history from growing past the limit.

```
# src/supportdesk/main.py
import sys

from supportdesk.config import load_config
from supportdesk.exceptions import MissingConfigError
from supportdesk.models.llm import (
    check_api_key_at_startup,
    create_client,
    stream_message,
)
# new in Step 2: trim_history and load_prompt
from supportdesk.utils.history import trim_history
from supportdesk.utils.logger import get_logger
from supportdesk.utils.prompts import load_prompt

# new in Step 2
# why: far below gpt-4o-mini's real limit (128,000 tokens), so the rough
# 4-characters-per-token guess can be off and still be safe
MAX_HISTORY_TOKENS = 3000

def main() -> None:
    # why: config first – a missing OPENAI_API_KEY stops the app here,
    # with one clear line, before anything else runs
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
```

```

client = create_client(config)
if not check_api_key_at_startup(client):
    logger.error("Authentication failed – check OPENAI_API_KEY.")
    print("Login failed – check OPENAI_API_KEY in your .env file.")
    sys.exit(1)

# new in Step 2: from here down – the loop replaces the one fixed call
# why: history[0] is the system prompt; every turn is appended after it
history = [{"role": "system", "content": load_prompt("concierge")}]
print("SupportDesk AI – type a message, or /quit to exit.")

while True:
    message = input("You: ").strip()
    if message == "/quit":
        break
    if message == "":
        # why: an empty line would cost an API call for nothing
        continue

    # when: before every call, so the request always fits the budget
    trim_history(history, MAX_HISTORY_TOKENS)
    logger.debug("Sending %d earlier messages", len(history))
    print("Assistant: ", end="", flush=True)
    stream_message(client, history, message)

if __name__ == "__main__":
    main()

```

Story — `tests/unit/test_history.py` : proves trimming keeps the system prompt and the newest turn, with no API key and no network. It uses the same plain `print` style as Doc04's test file. **If not:** a trimming bug would only show up as a strange reply after a long chat — very hard to trace back here.

```

# tests/unit/test_history.py
# Checks the history helpers. No API calls, no API key needed.
# Run from the repo root: python tests/unit/test_history.py
from supportdesk.utils.history import estimate_tokens, trim_history

def test_estimate_tokens() -> None:
    # 40 characters in total -> 40 // 4 = 10 tokens
    history = [
        {"role": "system", "content": "a" * 20},
        {"role": "user", "content": "b" * 20},
    ]

```

```

print(f"case 1 (estimate)           -> {estimate_tokens(history)}")

def test_short_history_untouched() -> None:
    history = [
        {"role": "system", "content": "be nice"},
        {"role": "user", "content": "hi"},
    ]
    trim_history(history, max_tokens=100)
    print(f"case 2 (short history)     -> {len(history)} messages kept")

def test_long_history_trimmed() -> None:
    # every turn is 400 characters (~100 tokens); the budget is 250
    history = [{"role": "system", "content": "be nice"}]
    for number in range(1, 6):
        history.append({"role": "user", "content": str(number) * 400})
    trim_history(history, max_tokens=250)
    first = history[0]["role"]
    newest = history[-1]["content"][0]
    print(f"case 3 (system prompt kept) -> {first}")
    print(f"case 3 (newest turn kept)     -> turn {newest}")
    print(f"case 3 (messages left)         -> {len(history)}")

if __name__ == "__main__":
    test_estimate_tokens()
    test_short_history_untouched()
    test_long_history_trimmed()

```

Run the test: `python tests/unit/test_history.py`

```

case 1 (estimate)           -> 10
case 2 (short history)     -> 2 messages kept
case 3 (system prompt kept) -> system
case 3 (newest turn kept)   -> turn 5
case 3 (messages left)     -> 3

```

Case 3 keeps 3 messages: the system prompt plus turns 4 and 5 (about 200 tokens — adding turn 3 would pass 250).

Run the app: `python -m supportdesk.main`, then:

```

SupportDesk AI – type a message, or /quit to exit.
You: my name is Ali
Assistant: Nice to meet you, Ali! How can I help you today?

```

```
You: what's the capital of France?  
Assistant: The capital of France is Paris.  
You: what's my name?  
Assistant: Your name is Ali.  
You: /quit
```

Difference from Basic: the prompt lives in its own file; `trim_history()` keeps every request under a budget without ever dropping the system prompt; empty lines are skipped; and a unit test proves the trimming rule without spending anything.

Which one should you actually write? Intermediate. Its trimming is what the checklist's "survives a too-long conversation" needs, and Step 4's Concierge agent moves this exact code into its own file.